MSAAD-Robustness — Setup Guide (Windows)

Shirish Giroti (CS23B2041), IIITDM Kancheepuram

30 Oct 2026

Before you start

- **Every command runs in a normal (non-admin) PowerShell window.** Nothing here needs Administrator rights, installs anything system-wide, or stores any API token.
- Everything Python-related lives inside one conda environment called `msaad`.
- Disk: about 1–2 GB free (one IoTID20 CSV + one CICIoT2023 CSV + the environment).
- Downloads are done in your browser. A free Kaggle account is only needed if you use the Kaggle mirror for CICIoT2023 (Step 5, option B).
- Each command block below says where to run it. Run the lines one at a time.

Step 1 — Put the code in a project folder

PowerShell (Windows):

```
New-Item -ItemType Directory -Force -Path "$HOME\Projects\CNS-Project" | Out-Null
Expand-Archive "$HOME\Downloads\msaad-robustness-v14c.zip" -DestinationPath "$HOME\Projects\CNS-Project" -Force
Set-Location "$HOME\Projects\CNS-Project\msaad-robustness"
Get-ChildItem
```

You should see `configs`, `experiments`, `scripts`, `src`, `tests` and the `.md` files. (If you prefer another location, change `$HOME\Projects\CNS-Project` everywhere.)

Updating an existing copy (e.g. v14b → v14c): run the same `Expand-Archive` line. The zip holds only code and documents, so your `data` and `results` folders are kept as they are. Once Step 15 is done, push each new version: `python scripts\push_run.py "Code v13: ..."`; git then shows exactly what changed. From v13 on the packages leave out `RUNS.md`, so extracting one never overwrites your journal.

Step 2 — Create the Python environment

PowerShell (Windows), inside the `msaad-robustness` folder:

```
conda create -n msaad -c conda-forge --override-channels python=3.11 -y
conda activate msaad
python --version
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

`python --version` must print `Python 3.11.x`. The `-c conda-forge --override-channels` flags avoid the Anaconda default-channel terms prompt. River ships a ready-made Windows wheel for Python 3.11, so no compiler is needed.

Every new PowerShell window: run `conda activate msaad` and `Set-Location "$HOME\Projects\CNS-Project\msaad-robustness"` before anything else.

Step 3 — Prove the pipeline works (no data needed)

PowerShell (Windows), `msaad` active:

```
python tests\smoke_test.py
```

Expected last line: `[smoke] PASS (>= 90.0)`. It streams 6,000 synthetic samples with concept drift through all five online learners, so give it a minute or two.

Step 4 — Download IoTID20 (one CSV)

Get the right file — this is the common trap: you need the **CSV of flow features** (~625k rows, columns ending in `Label`, `Cat`, `Sub_Cat`). The IEEE DataPort entry *"IoT Network Intrusion Dataset"* (an 823 MB zip) is the **raw PCAP** version — **do not use it**.

Browser: use the Kaggle mirror that published IoTID20 work uses (free Kaggle login): https://www.kaggle.com/datasets/rohulaminlabid/iotid20-dataset → **Download**. You get a `.zip` containing the CSV.

Do **not** use the "IoT-DI20" Google Drive folder on the Ontario Tech datasets page: it holds `IoT-23[Profile].csv` (about 22 MB), which is a different dataset.

PowerShell (Windows) — once the download has finished, pick the newest zip. Don't rely on the name: if Downloads already has an `archive.zip`, the browser saves the new one as `archive (1).zip`.

```
$zip = Get-ChildItem "$HOME\Downloads\*.zip" | Sort-Object LastWriteTime -Descending | Select-Object -First 1
$zip | Select-Object Name, Length, LastWriteTime
```

Check it is the file you just downloaded (its time should be a few minutes ago). Unpack it into a fresh folder and list everything inside:

```
Remove-Item "$HOME\Downloads\iotid20_raw" -Recurse -Force -ErrorAction SilentlyContinue
Expand-Archive -LiteralPath $zip.FullName -DestinationPath "$HOME\Downloads\iotid20_raw" -Force
Get-ChildItem "$HOME\Downloads\iotid20_raw" -Recurse -File | Select-Object Name, Length
```

Move the CSV into the project under the expected name:

```
New-Item -ItemType Directory -Force -Path ".\data\iotid20" | Out-Null
$csv = Get-ChildItem "$HOME\Downloads\iotid20_raw" -Recurse -Filter *.csv | Select-Object -First 1
if ($csv) {
    Move-Item -LiteralPath $csv.FullName ".\data\iotid20\IoT_Network_Intrusion_Dataset.csv"
    "Moved: $($csv.Name)"
} else { "No CSV in this zip - send Claude the listing above" }
```

Correct file check: Step 6's report should show about **625,783 rows**, about 86 columns, and the `Label` (Anomaly / Normal), `Cat` and `Sub_Cat` columns.

Step 5 — Download CICIoT2023 (ONE CSV is enough for now)

Do **not** download the whole dataset or the PCAP folder; they are very large.

Browser — pick one option:

- **A. Official (UNB CIC):** https://www.unb.ca/cic/datasets/iotdataset-2023.html → "Download the dataset" → `CICIoT2023` → `CSV` folder → download **any single** `.csv` . That server uses plain HTTP, so your browser may warn; option B is HTTPS.
- **B. Kaggle mirror (HTTPS, free account):** https://www.kaggle.com/datasets/madhavmalhotra/unb-cic-iot-dataset → in the file list, open the CSV folder and download **one** CSV file (not "Download All").

PowerShell (Windows), inside `msaad-robustness` — case 1: you downloaded ONE file. Kaggle gives a `.csv.zip` , the official site a plain `.csv` ; both are named `part-…` . Selecting by that name (not "newest zip") means this can't pick up the msaad package, the IoTID20 archive, or a half-finished download by mistake:

```
New-Item -ItemType Directory -Force -Path ".\data\ciciot2023" | Out-Null
$dl = Get-ChildItem "$HOME\Downloads\part-*" -File | Where-Object { $_.Extension -in ".zip", ".csv" } |
    Sort-Object LastWriteTime -Descending | Select-Object -First 1
$dl | Select-Object Name, Length, LastWriteTime
if ($dl -and $dl.Extension -eq ".zip") {
    Remove-Item "$HOME\Downloads\ciciot_raw" -Recurse -Force -ErrorAction SilentlyContinue
    Expand-Archive -LiteralPath $dl.FullName -DestinationPath "$HOME\Downloads\ciciot_raw" -Force
    $csv = Get-ChildItem "$HOME\Downloads\ciciot_raw" -Recurse -Filter *.csv | Select-Object -First 1
} else { $csv = $dl }
if ($csv) {
    Move-Item -LiteralPath $csv.FullName ".\data\ciciot2023\part-00000.csv" -Force
    "Moved: $($csv.Name)"
} else { "No finished CICIoT2023 download (part-*.zip or part-*.csv) in Downloads" }
```

PowerShell (Windows), inside `msaad-robustness` — case 2: a whole-dataset zip (e.g. Kaggle's "Download" button gives `archive.zip` , `archive (3).zip` , …). This lists the newest zip in Downloads that is not the msaad package and extracts ONE CSV between 20 MB and 1.5 GB, without unpacking the rest:

```
Add-Type -AssemblyName System.IO.Compression.FileSystem
$zf = Get-ChildItem "$HOME\Downloads\*.zip" | Where-Object { $_.Name -notlike "msaad-robustness*" } |
    Sort-Object LastWriteTime -Descending | Select-Object -First 1
"zip: $($zf.Name)  ($([math]::Round($zf.Length / 1MB)) MB)"
$z = [System.IO.Compression.ZipFile]::OpenRead($zf.FullName)
$files = $z.Entries | Where-Object { $_.Name }
"files in zip: $($files.Count)"
$files | Select-Object -Property FullName, @{n="MB"; e={[math]::Round($_.Length / 1MB, 1)}} -First 25 |
    Format-Table -AutoSize
$entry = $files | Where-Object { $_.Name -like "*.csv" -and $_.Name -notlike "IoT_Network*" } |
    Where-Object { $_.Length -gt 20MB -and $_.Length -lt 1.5GB } | Sort-Object FullName | Select-Object -First 1
if ($entry) {
    "extracting: $($entry.FullName)"
    New-Item -ItemType Directory -Force -Path ".\data\ciciot2023" | Out-Null
    [System.IO.Compression.ZipFileExtensions]::ExtractToFile($entry, (Join-Path $PWD.Path "data\ciciot2023\part-00000.csv"), $true)
} else { "no suitable CSV in this zip - send Claude the listing above" }
$z.Dispose()
```

If the zip printed on the first line is not the CICIoT2023 one, the download has not finished yet (or it is older than another zip); wait for it, or send Claude the listing.

Check both files are in place:

```
Get-ChildItem .\data -Recurse -File | Select-Object FullName, Length
```

Step 6 — Inspect both datasets and send me the reports

PowerShell (Windows), `msaad` active, inside `msaad-robustness` :

```
python scripts\inspect_dataset.py ".\data\iotid20\IoT_Network_Intrusion_Dataset.csv"
python scripts\inspect_dataset.py ".\data\ciciot2023\part-00000.csv"
Get-ChildItem .\results\inspect_*.txt
```

Each run prints the columns and the exact class counts, and saves a report in `results\` . **Paste both reports to Claude (or attach the two `.txt` files).** With them I confirm the label columns and the DDoS + benign subset so the reproduction matches the paper.

Step 7 — First real reproduction run

You can try this right after Step 6. If a column name differs from the defaults, the script stops with a message listing the real columns — just send me the reports.

PowerShell (Windows), `msaad` active:

```
python experiments\repro_baseline.py --data iotid20
python experiments\repro_baseline.py --data ciciot2023
```

Each run streams 50,000 rows of the DDoS + benign subset, taken evenly spaced across the whole file in file order, through the five learners. Expect roughly 10–30 minutes per dataset on your laptop (the progress bar shows samples/second). Results print at the end and are saved to `results\repro_<dataset>.json` . **Gate:** clean

accuracy within about 1–2 points of the paper (IoTID20 99.50%, CICIoT2023 99.28%).

Temporal protocol P2 (IoTID20 only). The public IoTID20 CSV is randomly shuffled with respect to time, so the runs above replay a stationary stream (P1 — this mirrors the paper). P2 replays the same rows with the attack flows in their real capture order (UDP flood first, then mixed UDP/ACK/HTTP floods) and the benign flows spread evenly through the stream as background traffic. **PowerShell (Windows):**

```
python experiments\repro_baseline.py --data iotid20_campaign
```

This loads the whole CSV once to sort it (about a minute extra). CICIoT2023's CSV has no timestamp column, so it only supports file order.

Do not use `iotid20_time` as a drift test. All 40,073 benign flows in IoTID20 were captured in its first week, so a plain time sort gives all-benign followed by all-attack: one class switch, with misleadingly perfect scores. It stays in the config only to document that finding.

IoTID20 subset sensitivity (optional). The paper lists Mirai ACK, *brute force*, HTTP and UDP flooding as IoTID20's DDoS types. `iotid20_mirai` keeps Normal + all four Mirai sub-categories, and `iotid20_all` keeps every attack (Normal vs Anomaly). Running them shows whether the clean result depends on the subset choice.

Step 8 — Paper project on Overleaf (browser)

Overleaf → New Project → **Upload Project** → `MSAAD-paper-overleaf.zip` → Settings → Compiler **pdfLaTeX**. It compiles to the skeleton with red `[TODO]` markers.

Step 9 — Run the experiment grid in parallel (e.g. overnight)

One run uses one CPU core and takes about 5–12 minutes for 50,000 IoTID20 rows (more when the laptop is busy or on battery), so run the whole grid in parallel. **PowerShell (Windows), `msaad` active, inside `msaad-robustness`:**

```
python experiments\run_grid.py --data iotid20 iotid20_campaign ciciot2023 --seeds 1 2 3 --jobs 4
```

- `--jobs 4` runs four at a time (safe with 16 GB RAM; the campaign-ordered runs hold the sorted file in memory). Datasets whose CSV is missing are skipped with a message.
- Add `iotid20_mirai iotid20_all` to the `--data` list for the subset sensitivity runs.
- Each result is saved as `results\grid\<dataset>_seed<k>.json`, with a log in `results\grid\logs\`. Every result also records the exact settings and library versions.
- If it stops (sleep, Ctrl+C), run the same command again: finished runs are skipped.
- At the end it prints mean ± std per dataset and writes `results\grid\summary.csv` — send that file to Claude.
- Keep the laptop plugged in and stop it from sleeping while the grid runs.

Step 10 — Under attack: the paper's Scenario 2, done properly

`experiments\under_attack.py` trains MABTSE on the first two thirds of a 30,000-row stream (as Algorithm 1 does), then replays the last third while a surrogate-based attacker perturbs it with FGSM, BIM, PGD and C&W (eps = 0.1, as in the paper). It reports the paper's metrics plus end-to-end ones (attack success rate, evasion gain, benign flips), audits how many adversarial vectors are impossible as real traffic, and re-runs each attack restricted to its feasible part. Everything runs offline on the dataset CSVs.

1. Check the attack stack (about a minute). PowerShell (Windows), `msaad` active:

```
python tests\attack_smoke_test.py
```

Expected last line: `[attack-smoke] PASS`.

2. Quick look on the real data (about 5–10 minutes):

```
python experiments\under_attack.py --data iotid20 --limit 3000
```

3. The full runs, three seeds in parallel (about 1–2 hours per run):

```
python experiments\run_grid.py --exp attack --data iotid20 ciciot2023 --seeds 1 2 3 --jobs 3
```

- Results: `results\attack_grid\<dataset>_seed<k>.json` and `.csv`, logs in `results\attack_grid\logs\`, and the mean ± std table in `results\attack_grid\summary_attack.csv` — send that file to Claude.
- Each run covers: clean, each attack on DDoS flows only (`attack_only`, the realistic threat model), on every flow (`all`, paper-style), and its feasible version; each with learning on ("online": true labels arrive instantly, as the paper assumes) and off ("frozen": no labels during the test). v7 skipped the frozen runs of the `all` scenarios; v8 includes them, and Step 11 fills them in for PGD and C&W without redoing a v7 grid.
- Every scenario also logs how many drift detections the attack triggered (`drifts_total`).
- Faster variant (about 45 minutes per run): add `--limit 20000` to the command.

Step 11 — Label feedback: delayed and self-labelled learning

The paper assumes the true label of every flow arrives the instant after it is classified. `under_attack.py --learning` replays the same attacks under other label settings:

Mode	Meaning
`online`	true label at once (the paper's Algorithm 1)
`frozen`	no labels during the test
`delay:D`	the true label of each flow arrives D flows later (e.g. `delay:1000`)
`self`	no labels: the model learns its own prediction (self-training)

Delayed and self-labelled runs cost as much as online ones, so they cover the clean and the raw `attack_only` scenarios only, and this grid uses the two strongest attacks (PGD and C&W). **PowerShell (Windows), `msaad` active, inside `msaad-robustness`** (about 1–1.5 hours per run, three at a time):

```
$modes = "frozen", "delay:100", "delay:1000", "delay:5000", "self"
python experiments\run_grid.py --exp attack --data iotid20 --jobs 3 --tag labels --attacks pgd cw --learning $modes
```

The first line only stores the list of label modes; PowerShell passes each one to the script as a separate option value.

- `--tag labels` keeps these results apart from Step 10's: files are `results\attack_grid\<dataset>_seed<k>_labels.json`, the summary is `results\attack_grid\summary_attack_labels.csv` — send that file to Claude.
- `frozen` is included so the `all` -mode frozen runs missing from a v7 grid get filled in.
- The test stream holds about 9,900 flows, so `delay:5000` means no feedback at all for the first half. At flood rates (thousands of flows per second) a label that takes even a few seconds to arrive is a delay of many thousands of flows.
- Drop `--attacks pgd cw` to cover all four attacks (about 1.5 times as long). Any other `under_attack.py` option can be added the same way.

Step 12 — RADP: the paper's Layer 1 (detect and delete), stress-tested

`experiments\radp_defense.py` rebuilds the paper's first defence layer and tests it against the same attacks as Step 10 (same seed = identical perturbations):

- **MSAPG:** the defender crafts FGSM/BIM/PGD/C&W samples on its own stand-in model, from its clean training flows only.
- **GAN:** a small NumPy GAN learns to mimic those samples (no PyTorch needed).
- **RAD:** a fresh copy of the online ensemble learns "clean vs adversarial".
- **Purification:** every flow RAD flags is deleted before the DDoS detector sees it.

It reports how many perturbed DDoS flows RAD catches, how many benign flows it deletes, the end-to-end attack success, the paper's "recovered accuracy" (survivors only) and accuracy when deleted flows count as blocked. It also runs RAD without the GAN (ablation) and "leave one attack out" (RAD never trained on the attack it faces: the paper's "unknown attacks" claim), and tests RAD on the repaired, realistic versions of the attacks.

1. Self-check (about 3 minutes). PowerShell (Windows), `msaad` active:

```
python tests\radp_smoke_test.py
```

Expected last line: `[radp-smoke] PASS`.

2. Quick look on the real data (about 10–15 minutes):

```
python experiments\radp_defense.py --data iotid20 --limit 3000
```

3. The full runs, three seeds in parallel (about 40 minutes per IoTID20 seed and 15 per CICIoT2023 seed, three at a time):

```
python experiments\run_grid.py --exp radp --data iotid20 --seeds 1 2 3 --jobs 3
```

- Results: `results\radp_grid\<dataset>_seed<k>.json` and `.csv`; the mean ± std table is `results\radp_grid\summary_radp.csv` — send that file to Claude.

Step 13 — S4: the adaptation loop under label poisoning, and the defence

`experiments\adaptation_loop.py` asks what happens when the labels MABTSE learns from can be fooled. The attacker's own DDoS flows are perturbed (the repaired, valid PGD of Step 10) and the labelling system calls each of them benign with probability q (`poison:q`; `true` means q = 0, the paper's Algorithm 1). Four update policies are compared:

Policy	What the defender does
`vanilla`	the paper: every label trains the learners and the Thompson-Sampling selector
`audit:a`	only a verified random share a of labels steers the selector; learners learn all
`guard:a`	audit + a "benign" label for a flow just flagged as DDoS needs verification
`strict:a`	audit + no unverified "benign" label is ever learned (DDoS labels are)
`iwstrict:a`	strict + each verified benign flow is learned round(1/a) times (at most 20), to give back the benign weight strict drops (v10)

It reports the evasion rate overall and per tenth of the stream, the false-alarm rate (the cost of each policy), how often each learner is selected (selector steering), and what each learner alone lets through. It also runs "poisoning only" (no perturbation) and a no-learning reference.

1. Self-check (about a minute). PowerShell (Windows), `msaad` active, inside `msaad-robustness`:

```
python tests\loop_smoke_test.py
```

Expected last line: `[loop-smoke] PASS`.

2. Quick look on the real data (about 5–10 minutes):

```
python experiments\adaptation_loop.py --data iotid20 --limit 3000
```

3. The full runs, three seeds in parallel (about 40 minutes per IoTID20 seed and 15 per CICIoT2023 seed, three at a time):

```
python experiments\run_grid.py --exp loop --data iotid20 ciciot2023 --jobs 3
```

- Results: `results\loop_grid\<dataset>_seed<k>.json` and `.csv`; the mean ± std table is `results\loop_grid\summary_loop.csv` — send that file to Claude.
- Other settings: `--labels true poison:0.25 poison:1.0`, `--policies vanilla strict:0.05`, `--variants raw feasible` (add the raw attack) or `--attacks pgd cw`.

4. The sweep (done on 1 Nov): smaller q and other verified shares. Same terminal:

```
$labels = "true", "poison:0.1", "poison:0.25", "poison:1.0"
$pols = "vanilla", "strict:0.01", "strict:0.05", "strict:0.2"
python experiments\run_grid.py --exp loop --data iotid20 ciciot2023 --jobs 3 --tag sweep --labels $labels --policies $pols
```

Results: `results\loop_grid\<dataset>_seed<k>_sweep.json` and `summary_loop_sweep.csv`.

5. IW-strict (v10, about an hour; done on 2 Nov, finding F14): can re-weighting the verified benign flows remove strict's false-alarm cost? PowerShell (Windows), `msaad` active, inside `msaad-robustness`:

```
python tests\loop_smoke_test.py
$pols = "iwstrict:0.05", "iwstrict:0.1", "iwstrict:0.2"
python experiments\run_grid.py --exp loop --data iotid20 ciciot2023 --jobs 3 --tag iw --policies $pols
```

- The self-check must end with `[loop-smoke] PASS` (it now also tests `iwstrict`).
- The `$pols` line only stores the three policies; PowerShell passes each one to the script.
- Results: `results\loop_grid\<dataset>_seed<k>_iw.json` and `summary_loop_iw.csv`. strict at the same shares is already in the main grid and the sweep (same seeds and audit masks), so it is not re-run.
- Then push it (Step 15; add `--push` to the grid command and it happens by itself), or zip the whole folder for Claude (it includes the sweep's per-seed files too):

```
Compress-Archive -Path .\results\loop_grid -DestinationPath "$HOME\Downloads\msaad_loop.zip" -Force
```

Step 14 — Figures and tables for the paper

`scripts\make_figures.py` turns the grid summaries into the paper's figures (PDF for Overleaf, PNG to look at) and LaTeX tables, mean ± std over seeds. It skips whatever has not been run yet, so run it again whenever new results arrive.

One-time install (about 10 MB). PowerShell (Windows), `msaad` active:

```
python -m pip install matplotlib
```

Make the figures and tables. PowerShell (Windows), `msaad` active, inside `msaad-robustness`:

```
python scripts\make_figures.py
Get-ChildItem .\results\paper
```

- Figures: `fig_attack_asr`, `fig_label_delay`, `fig_radp_detect`, `fig_radp_e2e`, `fig_loop_poison`, `fig_loop_time`, `fig_audit_tradeoff` (`.pdf` + `.png`). Upload the PDFs to the Overleaf `images` folder.
- Bars and lines are means over seeds; each seed is a small dot. Where a grid's per-seed JSONs are missing, an error bar (± one standard deviation) is drawn instead.
- Tables: `tab_repro.tex`, `tab_attack.tex`, `tab_radp.tex`, `tab_loop.tex`, `tab_audit.tex`. Paste them into the paper and add `\usepackage{booktabs}` to the preamble. Tables with more than six columns are written as `table*` (full page width).
- After the IW-strict run (Step 13, part 5) run it again: IW-strict joins `fig_loop_poison`, `fig_loop_time`, `fig_audit_tradeoff` and the tables automatically.
- The table captions are short placeholders: rewrite them in your own words.

Step 15 — A private GitHub repository, pushed after every run

Every run's results, logs and a journal entry (`RUNS.md`) go to a private GitHub repository: the project history is backed up and every number in the paper can be traced to the run and code version that produced it. Datasets are never uploaded.

Least privilege: the laptop pushes with an SSH **deploy key**, a key that works for this one repository only and gives no access to the rest of your GitHub account. Nothing here needs administrator rights, and every `git config` below applies to this repository only.

1. Check git and ssh. PowerShell (Windows), `msaad` active:

```
git --version
ssh -V
```

If `git` is not found, install it into the msaad environment (no admin rights needed):

```
conda install -n msaad -c conda-forge --override-channels git -y
```

2. Create the empty repository (browser). github.com → **New repository** → name `msaad-robustness` → **Private** → leave *Add a README*, *.gitignore* and *license* unticked → **Create repository**.

3. Make the deploy key. PowerShell (Windows), any folder:

```
New-Item -ItemType Directory -Force -Path "$HOME\.ssh" | Out-Null
ssh-keygen -t ed25519 -f "$HOME\.ssh\msaad_deploy" -C "msaad-robustness laptop"
Get-Content "$HOME\.ssh\msaad_deploy.pub" | Set-Clipboard
```

- `ssh-keygen` asks for a passphrase: press Enter twice for none (the key opens only this repository), or set one and type it at every push.
- The last line copies the **public** key (`.pub`). On GitHub: the repository → **Settings** → **Deploy keys** → **Add deploy key** → title `laptop`, paste, tick **Allow write access** → **Add key**. The file without `.pub` is the private key: it never leaves your laptop.

4. Turn the project folder into the repository. PowerShell (Windows), `msaad` active, inside `msaad-robustness`:

```
git init -b main
git config user.name "Shirish Giroti"
git config user.email "YOUR-GITHUB-NOREPLY-ADDRESS"
$key = "$HOME/.ssh/msaad_deploy" -replace '\\', '/'
git config core.sshCommand "ssh -i $key -o IdentitiesOnly=yes"
git remote add origin git@github.com:dovahkkin1234/msaad-robustness.git
```

- For the e-mail, use your GitHub *noreply* address (GitHub → Settings → Emails), so your real address does not appear in the commits.
- `.gitignore` (in the zip) keeps `data\`, caches, archives and key files out of the repository.

5. The first push: everything done so far. Same terminal:

```
python scripts\push_run.py --dry-run
python scripts\push_run.py "Repository created: code v12, documents and all results up to 2 Nov"
```

- The dry run shows the journal entry it would write and how many files it would commit, and changes nothing. `data\` is never committed (`.gitignore`), and the script refuses any file over 50 MB or anything that looks like a key.
- On the first push ssh asks whether to trust github.com. Type `yes` only if the fingerprint it shows is one of GitHub's published ones: Ed25519 `SHA256:+DiY3wvvV6TuJJhbpZisF/zLDA0zPMSvHdkr4UvCOqU` or ECDSA `SHA256:p2QAMXNIC1TJYWeIOttrVc98/R1BUFWu3/LiyKgUfQM` .
- Open the repository page: the code, `results` , `RUNS.md` — and no `data` folder.

6. From now on, after every run. Add `--push` to the end of any grid command; when the grid ends it writes the journal entry, commits and pushes. For example, two more CICIoT2023 seeds for the main loop grid:

```
python experiments\run_grid.py --exp loop --data ciciot2023 --seeds 4 5 --jobs 2 --push
```

After any other run (a quick look, new figures, a new code version from Claude):

```
python scripts\push_run.py "Quick look: RADP on IoTID20, --limit 3000"
python scripts\push_run.py -m "IW-strict halves the false alarms on CICIoT2023"
```

- Each push adds an entry to `RUNS.md` (date, the exact command, code version, which runs worked, which result files changed, your note) and rebuilds the run-history table from the result JSONs. Every grid is also recorded in `results\grid_runs.jsonl` .
- It refuses to commit anything under `data\` , any file over 50 MB, or anything that looks like a private key, and it never rewrites history.
- Offline? The commit stays on the laptop; the next `push_run.py` sends it.

Step 16 — Adversarial-training baseline (the defence reviewers ask about first)

Goal. Without labels, repaired attacks get 80–87% of DDoS flows past MABTSE on CICIoT2023 (F10). The standard fix is adversarial training: the defender crafts adversarial copies of its own DDoS training flows (on its own surrogate, a different seed from the attacker's), repairs them to valid traffic, and trains MABTSE on them labelled DDoS. `experiments\adv_training.py` trains three detectors per run and attacks each with the same perturbations as Step 10:

Variant	Trained on	Question it answers
`none`	clean flows only (reproduces Step 10's numbers exactly)	the baseline
`at:pgd`	+ one repaired PGD copy per DDoS flow	does it carry over to attacks it never saw (FGSM, C&W; BIM is a near-copy of PGD)?
`at:fgsm+bim+pgd+cw`	+ one copy per DDoS flow, the four attacks in turn	best case: every attack seen

Each detector is tested with no labels (all four attacks, raw and repaired) and with labels 1,000 flows late (clean, repaired PGD and C&W), reporting the evasion rate and the false-alarm rate on clean traffic: the price of adversarial training.

1. Self-check (about 1 min). PowerShell (Windows), `(msaad)` active, inside `msaad-robustness` :

```
python tests\advtrain_smoke_test.py
```

It must end with `[advtrain-smoke] PASS` .

2. The grid: both datasets, 3 seeds, pushed when it ends. Same terminal:

```
python experiments\run_grid.py --exp advtrain --data iotid20 ciciot2023 --jobs 3 --push
```

- About 2.5–3.5 hours with `--jobs 3` (IoTID20 runs take about twice as long as CICIoT2023). Keep the laptop plugged in and stop it from sleeping (Settings → System → Power).
- Interrupted? Run the same line again: finished runs are skipped.
- Results go to `results\advtrain_grid` (one JSON + CSV per run, logs in `logs`), the summary to `summary_advtrain.csv` . Then `python scripts\make_figures.py` adds `fig_advtrain` and `tab_advtrain` to `results\paper` .

3. Send me `results\advtrain_grid\summary_advtrain.csv` and the terminal output (or just say it is pushed).

Step 17 — v14a: framing, audit allocation, low audit budgets, and paired statistics

Branch `exp/conference` only (check with `git branch --show-current`). v14a adds to `experiments\adaptation_loop.py` (v13 runs are reproduced bit for bit):

Option	What it tests
`--labels frame:r`	**Framing / availability poisoning through the trusted direction**: the attacker's bots replay benign-looking flows that are (correctly) labelled DDoS; every policy that trusts DDoS labels learns them. Measured: false alarms on genuine benign traffic during and after the campaign. Combine with `+` , e.g. `poison:1.0+frame:0.05` .
`--policies consensus:a` , `iwconsensus:a`	The defence against framing: an unverified DDoS label is learned only when the detector also says DDoS. Its price: slower adaptation to evasion.
`--policies iwstrict:a:c`	IW-strict with at most c copies (weight clipping) for small audit budgets.
`--policies noiseaware:a`	A trusted-audit noise-rate baseline (GLC-style): learns each unverified benign label with probability 1 - rho, rho estimated from audits.
`--audits uncertain` / `mixed`	Audits chosen by the detector's uncertainty (or half random, half uncertain) instead of at random. An evading attacker's flows look confidently benign, so uncertainty-only audits never check them.

`scripts\stats.py` turns the per-seed results into paired statistics (bootstrap CIs, Wilcoxon signed-rank, Holm correction, effect sizes) for every headline comparison.

1. Self-check. PowerShell (Windows), `(msaad)` active, inside `msaad-robustness` :

```
python tests\loop_smoke_test.py
```

2. The four v14a grids, 15 seeds each, pushed when each ends. Same terminal:

```
$s = 1..15
$d = "iotid20", "ciciot2023"
$g = "--exp", "loop", "--data"
$lab = "true", "frame:0.02", "frame:0.1", "poison:1.0+frame:0.05"
$pol = "vanilla", "strict:0.1", "iwstrict:0.1", "consensus:0.1", "iwconsensus:0.1"
python experiments\run_grid.py $g $d --seeds $s --jobs 6 --tag frame --labels $lab --policies $pol --push
$lab = "true", "poison:1.0"
$pol = "strict:0.1", "iwstrict:0.1"
python experiments\run_grid.py $g $d --seeds $s --jobs 6 --tag audits --audits uncertain --labels $lab --policies $pol --push
python experiments\run_grid.py $g $d --seeds $s --jobs 6 --tag auditsmix --audits mixed --labels $lab --policies $pol --push
$pol = "strict:0.1", "noiseaware:0.1", "strict:0.05", "iwstrict:0.05", "iwstrict:0.05:5"
$pol += "iwstrict:0.05:10", "strict:0.02", "iwstrict:0.02:10"
python experiments\run_grid.py $g $d --seeds $s --jobs 6 --tag lowbudget --labels $lab --policies $pol --push
```

PowerShell passes each element of an array variable (`$d` , `$lab` , `$pol`) as its own argument.

3. Statistics for everything that has finished. Same terminal:

```
python scripts\stats.py
python scripts\push_run.py "Paired statistics (15 seeds)"
```

Tables land in `results\paper\stats_*.csv` and `stats_*.tex` .

Step 18 — v14b: the chain of detectors, adaptive attackers, and the third dataset

Branch `exp/conference` only. Needs PyTorch (CPU build; it does not touch River or numpy).

File	What it does
`experiments\cascade_eval.py`	**Chain of detectors**: MABTSE (stage 1, identical to the attack grid), an online MLP (stage 2) and a host-graph neural network (stage 3) in three versions: `gnn_full` , `gnn_drift` (only the 2-hop neighbourhoods of hosts whose traffic just changed) and `ctx` (the matched control without message passing). A later stage only re-checks flows the earlier ones let through. Also a graph-aware attacker: `spoof:k` (spread over many sources) and `thin:rho` (send fewer flows).
`src\models\online_mlp.py`	The online MLP (PyTorch, River-style).
`src\models\graph_stage.py`	Window host graphs, the graph-attention network, the matched control, per-host drift detection.
`src\attacks\decision.py`	Label-only HopSkipJump (queries only the detector's verdicts).
`experiments\adv_training.py --adaptive distill atsur hsj`	Adaptive attackers against the plain and adversarially trained detectors: a surrogate distilled from the detector's own verdicts, an adversarially trained surrogate, and HopSkipJump with raw and with valid-traffic queries.
`src\data\context.py`	Host addresses and timestamps for the graph and for label delays in seconds (`delaysec:T`).

1. Install PyTorch (once). PowerShell (Windows), `(msaad)` active:

```
python -m pip install torch --index-url https://download.pytorch.org/whl/cpu
```

2. Self-checks. PowerShell (Windows), `(msaad)` active, inside `msaad-robustness` :

```
python tests\cascade_smoke_test.py
python tests\smoke_test.py
```

Both must end with PASS.

3. The v14b grids. Same terminal:

```
$s = 1..15
$d = "iotid20_campaign", "ciciot2023"
python experiments\run_grid.py --exp cascade --data $d --seeds $s --jobs 6 --tag s15 --push
$v = "none", "at:pgd"
$a = "distill", "atsur", "hsj"
$c5 = "--data", "iotid20", "ciciot2023", "--seeds", "1", "2", "3", "4", "5", "--jobs", "6"
python experiments\run_grid.py --exp advtrain $c5 --tag adaptive --variants $v --learning frozen --adaptive $a --push
```

- The chain uses IoTID20 in campaign order (`iotid20_campaign`): the file order is shuffled, so its windows would not be time windows. CICIoT2023 has no host addresses: stages 1-2 only.
- Expect about 4 hours for the chain grid and 1.5 hours for the adaptive grid with 6 jobs.

4. The third dataset: corrected CIC-IDS2017 (Engelen et al., WTMC 2021). Download the improved CIC-IDS2017 from https://intrusion-detection.distrinet-research.be/WTMC2021/tools_datasets.html (the Friday file), put it at `data\cicids2017\friday.csv` , then run the inspector and send Claude the report (column names, label values and timestamp format are confirmed from it):

```
python scripts\inspect_dataset.py .\data\cicids2017\friday.csv
```

Done on 3 Nov; the report set the configuration. Continue with Step 19.

Step 19 — v14c: CIC-IDS2017, the third dataset

Branch `exp/conference` only. What v14c changes (IoTID20 and CICIoT2023 streams are identical with it, so extracting it while their grids run does not change their results):

File	Change
`configs\base.yaml`	`cicids2017` set from the inspector report: dropped columns, timestamp format, and a replay window of 20:31–21:26 around the flood (20:56:31–21:16:10 on the file's clock).
`experiments\under_attack.py` , `repro_baseline.py`	Now honour `time_window` and `exclude_values` . Before, only the chain did, so on CIC-IDS2017 the other scripts would have replayed the whole day.
`experiments\run_grid.py`	A summary now covers every dataset and seed with the grid's tag on disk, so a CIC-IDS2017 grid with `--tag s15` keeps the IoTID20 and CICIoT2023 rows. `--summarise` rewrites a summary without running anything.
`scripts\check_stream.py`	What a configured stream contains: the train/test split, classes per part, clock times, label delays in seconds, the attacking hosts. Trains nothing.
`scripts\per_seed.py`	The per-seed values behind one summary line, with the median, the IQR and how many seeds sit at either end.
`scripts\stats.py`	A `cascade` preset: each chain vs MABTSE alone, the GNN vs its matched control, the graph-aware attacker, strict vs vanilla inside the chain.
`scripts\make_figures.py`	Prefers the 15-seed results; adds the chain, framing, adaptive-attack, audit-allocation and low-budget figures and tables; one panel per dataset.

1. The data (done on 3 Nov). The WTMC 2021 download (`dataset.zip`) holds one CSV per weekday; only Friday is needed. PowerShell (Windows), `(msaad)` active, inside `msaad-robustness` ; this copies out the one file without unpacking the rest:

```
New-Item -ItemType Directory -Force -Path data\cicids2017 | Out-Null
Add-Type -AssemblyName System.IO.Compression.FileSystem
$z = [System.IO.Compression.ZipFile]::OpenRead("$HOME\Downloads\dataset.zip")
$e = $z.Entries | Where-Object { $_.Name -eq "Friday-WorkingHours.csv" }
[System.IO.Compression.ZipFileExtensions]::ExtractToFile($e, (Join-Path $PWD.Path "data\cicids2017\friday.csv"), $true)
$z.Dispose()
```

2. Update the code and self-check. Same kind of window (a new one if a grid is running):

```
Expand-Archive "$HOME\Downloads\msaad-robustness-v14c.zip" -DestinationPath "$HOME\Projects\CNS-Project" -Force
python tests\smoke_test.py
python tests\cascade_smoke_test.py
python scripts\push_run.py "Code v14c: CIC-IDS2017 configured, time-window fix, stream check, chain statistics"
```

3. Check the stream (about a minute):

```
python scripts\check_stream.py --data cicids2017
```

Expect 30,000 rows; the training part ending inside the flood (around 21:09); a test DDoS share of roughly 75–85%; `context loader: same rows and labels (OK)` ; every DDoS flow from 172.16.0.1; and `[check] OK` at the end. Send Claude the output.

4. The CIC-IDS2017 grids, 15 seeds, each pushed when it ends. Same window:

```
$s = 1..15
$c = "--data", "cicids2017", "--jobs", "6", "--tag", "s15", "--push"
$lrn = "online", "frozen", "delay:1000", "delay:5000", "delaysec:60", "delaysec:300"
$lab = "true", "poison:0.25", "poison:0.5", "poison:1.0"
$pol = "vanilla", "audit:0.1", "guard:0.1", "strict:0.1", "iwstrict:0.1", "iwstrict:0.2"
python experiments\run_grid.py --exp attack $c --seeds $s --attacks pgd cw --perturb attack_only --learning $lrn
python experiments\run_grid.py --exp loop $c --seeds $s --labels $lab --policies $pol
python experiments\run_grid.py --exp advtrain $c --seeds $s --variants none at:pgd
python experiments\run_grid.py --exp cascade $c --seeds $s
```

- The same settings as the IoTID20 and CICIoT2023 15-seed grids, plus label delays in seconds (`delaysec:60` , `delaysec:300` ; the test stream covers about 17 minutes of traffic).
- Next to another grid this means 12 runs at once. If Task Manager shows memory above about 85%, stop this window (Ctrl+C), set `$c[3] = "3"` (three runs at a time) and start the same line again: finished runs are skipped.
- On an idle laptop: about 1, 2.5, 1 and 2 hours with 6 jobs; longer next to another grid.

5. When every grid has finished: complete summaries, statistics and figures. Same window:

```
foreach ($e in "attack", "loop", "advtrain", "cascade") { python experiments\run_grid.py --exp $e --tag s15 --summarise }
python scripts\stats.py
python scripts\make_figures.py
python scripts\push_run.py "15-seed summaries, statistics and figures, three datasets"
```

`--summarise` matters when two grids with the same tag ran side by side: whichever finished last wrote the summary, and an older copy of `run_grid.py` leaves out datasets it did not run.

6. A result with a large spread, seed by seed (example: frozen transfer on CICIoT2023):

```
python scripts\per_seed.py --exp attack --tag s15 --data ciciot2023 --scenario pgd/attack_only/feasible --learning frozen
```

Troubleshooting

Symptom	Fix
`conda` is not recognized	Open **Anaconda PowerShell Prompt** from the Start menu and run the steps there. (`conda init powershell` also works, but it edits your PowerShell profile.)
Terms-of-service error on `conda create`	Keep the `-c conda-forge --override-channels` flags exactly as written.

`ModuleNotFoundError: river`	The env is not active: `conda activate msaad`.
`dataset file not found`	The file name/folder differs; run `Get-ChildItem .\data -Recurse -File` and compare with Steps 4–5.
`KeyError: column(s) ... not in ...`	The message lists the real columns; send me the Step 6 reports.
Very slow run	Lower `limit` in `configs\base.yaml` (e.g. 20000) for a first test.
`the surrogate needs both classes`	The training part of the stream had only one class; use `iotid20` or `iotid20_campaign`, not `iotid20_time`.
`ConvergenceWarning` from scikit-learn	Harmless: the surrogate MLP stops early by design.
`options [...] are not used by the repro experiment`	`--learning`, `--attacks`, `--labels` etc. only apply with `--exp attack`, `--exp radp` or `--exp loop`.
`unknown --learning mode`	Use `online`, `frozen`, `self` or `delay:D` with a whole number, e.g. `delay:1000`.
`unknown --labels value` / `unknown --policies value`	Use `true` or `poison:q`, and `vanilla`, `audit:a`, `guard:a`, `strict:a` or `iwstrict:a`, with q and a between 0 and 1 (e.g. `poison:0.5`, `strict:0.1`). `iwstrict` needs v10.
`ModuleNotFoundError: No module named 'torch'`	Step 18, part 1 (CPU build of PyTorch).
`delaysec:T needs ...`	Label delays in seconds need a time-ordered dataset with a timestamp in its `context` config (CIC-IDS2017), and v14c (older code loads different rows for the timestamps).
`[check] ... the context loader returns different rows`	The code is older than v14c: extract v14c (Step 19, part 2).
`[grid] --data is required (or use --summarise)`	Add `--data ...`, or `--summarise` to only rewrite a summary from finished runs.
A summary CSV lacks a dataset you ran	Two grids with the same tag finished in the wrong order: `python experiments\run_grid.py --exp <exp> --tag <tag> --summarise`.
`unknown variant` (adv_training.py)	Use `none` or `at:` followed by attacks joined with `+`, e.g. `at:pgd`, `at:fgsm+bim+pgd+cw`.
`ModuleNotFoundError: No module named 'matplotlib'`	Run the one-time install in Step 14.
`[push] git is not installed or not on PATH`	Step 15, part 1.
`Permission denied (publickey)` on push	The deploy key is not on the repository, or `core.sshCommand` points elsewhere: check Step 15, parts 3–4.
`The key you are authenticating with has been marked as read only`	Delete the deploy key on GitHub and add it again with **Allow write access** ticked.
`[push] refusing to commit: ...`	The message names the file (data, over 50 MB, or a key): move it out of the project folder and re-run.
`rejected ... (fetch first)` on push	GitHub has commits this laptop lacks: `git pull --rebase`, then re-run `python scripts\push_run.py`.
`ssh-keygen` / `ssh` is not recognized	Windows 10/11 ship the OpenSSH client (Settings → System → Optional features); Git for Windows also includes it.